AVX2 Implementation of QR-UOV for Modern x86 Processors

Hiroshi Amagasa[1], Rei Ueno[2,1] and Naofumi Homma[1]

[1] Research Institute of Electrical Communication/Graduate School of Engineering, Tohoku University, 2–1–1 Katahira, Aoba-ku, Sendai-shi, 980-8577, Japan, hiroshi.amagasa.q7@dc.tohoku.ac.jp, naofumi.homma.c8@tohoku.ac.jp

[2] Graduate School of Informatics, Kyoto University, Yoshida-honmachi, Sakyo-ku, Kyoto, 606-8501, Japan, ueno.rei.2e@kyoto-u.ac.jp

Abstract. QR-UOV is a multivariate signature scheme selected as one of the candidates in the second round of the NIST PQC Additional Digital Signatures process. This paper presents software acceleration methods for QR-UOV optimized for modern x86 architectures. QR-UOV operates over small odd prime-power extension fields such as GF(31^3) and GF(127^3) unlike other multivariate signature candidates. This property allows direct utilization of hardware multipliers for field arithmetic, offering a distinctive advantage for high-performance implementations. Yet, how to implement QR-UOV efficiently on modern CPUs based on the property remains unclear so far. Our implementation benefits from two proposed ideas: (1) reducing the computational overhead of the QR-UOV algorithm through algorithm-level optimization, and (2) leveraging advanced SIMD instruction set extensions (e.g., AVX2, AVX-512) to accelerate main operations such as matrix multiplication. Our implementation achieves substantial speedups over the Round 2 reference: for the parameter set $(q, \ell) = (127, 3)$ at NIST security level I, it delivers a 5.1× improvement in key generation, 3.6× in signature generation, and 5.7× in signature verification. These results demonstrate that QR-UOV achieves performance comparable or higher than that of UOV implementations, particularly at higher security levels.

Keywords: Multivariate Cryptography · Oil and Vinegar · AVX2

1 Introduction

1.1 Background

NIST's Post-Quantum Cryptography (PQC) standardization initiative has progressed steadily since its inception in 2016, with three digital signature schemes selected in the initial round. To diversify its portfolio beyond lattice-based solutions, NIST called for additional digital signature proposals in 2022. From over 40 proposals evaluated in the first round, NIST announced that 14 candidates advanced to the second round. The second round candidates cover a diverse set of design families: two code-based, five MPC-in-the-Head, four multivariate (MQ)-based, one lattice-based, one isogeny-based, and one symmetric-based scheme. Among them, MQ-based signatures are based on the hardness of solving systems of multivariate quadratic equations over a finite field. They typically offer fast signature generation and verification, making them suitable for resource-constrained environments such as embedded systems. Moreover, their mathematical structure allows for relatively small computational footprints. In contrast, a common trade-off is the larger size of public keys and signatures compared to other schemes.

QR-UOV (Quotient Ring Unbalanced Oil and Vinegar) [AFI+24] is an MQ-based signature scheme and one of the second round candidates. It introduces a quotient-ring-based algebraic structure over a finite field, primarily to reduce the public key size compared to the conventional UOV. While there has been substantial work on optimizing some MQ-based signatures, optimization techniques specifically exploiting QR-UOV's quotient-ring arithmetic have received relatively little attention. Consequently, QR-UOV's practical performance remains less established beyond the provided reference implementation.

1.2 Our contributions

This paper describes optimization methods that implement the QR-UOV Round 2 submission algorithms efficiently on modern CPUs. Our contributions are detailed as follows:

- We propose algorithm-level methods that substantially reduce the computational complexity of core arithmetic operations, such as matrix multiplications, in the QR-UOV signature scheme. These methods enhance overall efficiency and are applicable to both software and hardware implementations, providing a versatile foundation for future performance improvements across diverse platforms.

- We develop efficient software acceleration techniques that fully leverage advanced instruction set extensions available in modern x86 processors, including AVX2 and AVX-512. These techniques enable high-speed execution of QR-UOV and achieve performance comparable to or even higher than that of conventional UOV, which underscores the practicality of QR-UOV for real-world applications.

- While most existing multivariate polynomial signature implementations focus on even-characteristic fields such as $\mathrm{GF}(2^4)$ or $\mathrm{GF}(2^8)$, this work explores optimization methods tailored to small odd prime-power fields, such as $\mathrm{GF}(31^3)$ and $\mathrm{GF}(127^3)$, used in QR-UOV. By utilizing integer multipliers and recent SIMD extensions, we propose efficient arithmetic methods that broaden the design space for MQ signature implementations.

1.3 Related work

In [CCC+09, CLP+18], implementation techniques for MQ cryptosystems were presented on contemporary x86 architectures using SIMD instructions. Instead of the traditional byte-oriented method, the works showed packing multiple field elements into SSE/AVX registers and performing parallel modular multiplications and additions using integer arithmetic units, thereby exploiting the CPU's vector processing capabilities. They also introduced optimized reduction methods tailored to small primes and discussed memory layout strategies to maximize data throughput. The results demonstrated significant speedups in both signature generation and verification compared to conventional scalar implementations, underscoring the potential of SIMD-based approaches for MQ cryptosystems. In our work, we focus on odd-prime QR-UOV and extend these techniques with support for modern inner-product instructions, notably AVX-512 VNNI.

In [BCH+23], Beullens et al. provided a comprehensive study of implementations for the UOV signature scheme across multiple platforms, including modern x86 CPUs, ARM Neon, Cortex-M4 microcontrollers, and FPGAs. They showed efficient implementations that leveraged platform-specific features, such as SIMD vectorization on x86 and ARM Neon, constant-time finite field arithmetic, and resource-efficient FPGA designs, to achieve performance improvements in both signing and verification. The results highlighted that with architecture-level optimization, UOV can offer competitive performance and compact implementations suitable for constrained embedded devices and high-performance systems.

MQDSS [SCH+19] was a candidate in the second round of the original NIST PQC process. It is a multivariate signature scheme defined over the odd modulus $q = 31$, similarly to QR-UOV. Chen et al. [CHR+16] presented an AVX2-optimized implementation of MQDSS that leverages the 8-bit integer multiply-add instruction, `VPMADDUBSW`. While their work focuses on optimizing MQ evaluation through a tailored coefficient layout, we primarily utilize `VPMADDUBSW` in key generation to emulate VNNI-style dot products within matrix multiplications.

On the other hand, all other multivariate signatures submitted to the NIST PQC standardization process, like MAYO [BCC+24], SNOVA [WCD+24], UOV [BCD+24], and former Rainbow [DCP+20] utilize $\mathrm{GF}(2^4)$ or $\mathrm{GF}(2^8)$, which makes it difficult to directly apply their optimization techniques to QR-UOV. We also note that the implementation techniques summarized in [Hwa24] for lattice-based cryptosystems that rely heavily on polynomial operations are not fully applicable to QR-UOV with small parameters $q = 7, 31, 127$ and $\ell = 3, 10$. While some of these techniques may be partially applicable to QR-UOV, the efficient acceleration of the scheme on modern CPUs, particularly for computations arising from its quotient-ring structure, remains an open problem.

1.4 Paper organization

Section 2 briefly introduces QR-UOV and its notations. Section 3 proposes algorithm-level implementation methods for QR-UOV. Section 4 presents architecture-level implementation methods specified for modern x86 processors such as SIMD vectorization (i.e. AVX2 and AVX-512). Section 5 shows the implementation results and compares it to other PQC signatures. Finally, Section 6 concludes this paper.

2 Preliminaries

2.1 Overview of QR-UOV

Multivariate public key cryptography is based on the hardness of solving a system of multivariate quadratic polynomial equations over a finite field, known as the multivariate quadratic (MQ) problem. The key generation algorithm generates the structured central quadratic map $\mathcal{F}$, and a random linear map $\mathcal{S}$. The private key is defined as a pair $(\mathcal{F}, \mathcal{S})$, and the public key is defined as $\mathcal{P} = \mathcal{F} \circ \mathcal{S}$. Here, $\circ$ denotes the function composition. In general, we cannot efficiently solve the simultaneous equations of the public key. In contrast, we can solve them using the private key. The signature s and the message m have the relationship $m = \mathcal{P}(s)$, and in the signature generation, we derive the signature s using the private key. We can verify the signature by checking whether the equation holds.

The unbalanced oil and vinegar (UOV) signature [KPG99] is a well-established MQ signature, where a system of quadratic equations is used with v vinegar variables and m oil variables such that $v > m$. Since the simultaneous quadratic equations are linear to the oil variables, after generating the vinegar variables at random, the signature (solution) can be generated by solving the linear equations to the oil variables. While UOV has the advantage of short signature size and fast verification, its major drawback is that the public key size is much larger than that of other signatures. Addressing the drawback, QR-UOV, a variant of UOV, was constructed by utilizing an algebraic structure.

QR-UOV introduces a structure of a quotient ring to the system of equations whereas the original UOV utilizes an unstructured system of equations, which enables compact representation of public keys and potentially more efficient implementations. Thus, QR-UOV leverages structured algebraic properties of the quotient ring to significantly reduce key sizes without compromising security. This structural change not only preserves the

core security assumptions of UOV but also opens new avenues for optimizing performance and reducing memory requirements in MQ signatures.

QR-UOV differs algebraically from other MQ signature schemes such as UOV, MAYO, and SNOVA primarily in its use of a quotient ring structure. While other schemes define their multivariate quadratic systems over extension fields of characteristic two, typically $\mathrm{GF}(2^4)$ or $\mathrm{GF}(2^8)$, QR-UOV operates over a quotient ring $\mathbb{F}_q[x]/(f(x))$, where $q \in \{7, 31, 127\}$ is a small odd prime and the degree of $f(x)$ is small (3 or 10), enabling the compact encoding of public key polynomials and the exploitation of ring-specific arithmetic properties. Thus, the quotient ring formulation in QR-UOV is its key distinguishing feature, combining UOV-like security assumptions with improved algebraic efficiency.

We first review the basics of UOV since QR-UOV is an extension of UOV. Let $\boldsymbol{x} = (x_1, \ldots, x_n) \in \mathbb{F}_q^n$, where $(x_1, \ldots, x_v)$ are vinegar variables and $(x_{v+1}, \ldots, x_n)$ are oil variables. The private key consists of $\mathcal{F} = (f_1, \ldots, f_m) : \mathbb{F}_q^n \to \mathbb{F}_q^m$, where each f_i is a quadratic polynomial, and a random linear map $\mathcal{S} : \mathbb{F}_q^n \to \mathbb{F}_q^n$. The public key is $\mathcal{P} = \mathcal{F} \circ \mathcal{S} : \mathbb{F}_q^n \to \mathbb{F}_q^m$. To generate a signature on $\boldsymbol{t} \in \mathbb{F}_q^m$, one fixes random vinegar variables $x_1, \ldots, x_v \in \mathbb{F}_q$, and solves the resulting linear system in oil variables $x_{v+1}, \ldots, x_n$ to obtain $\boldsymbol{x} \in \mathbb{F}_q^n$ such that $\mathcal{F}(\boldsymbol{x}) = \boldsymbol{t}$. The signature is then $\boldsymbol{s} = \mathcal{S}^{-1}(\boldsymbol{x})$. The verifier checks whether $\mathcal{P}(\boldsymbol{s}) = \boldsymbol{t}$. Each quadratic polynomial f_i of the central map $\mathcal{F}$ can be expressed by an $n \times n$ matrix F_i with block structure as

$$F_i = \begin{bmatrix} *_{v\times v} & *_{v\times m} \\ *_{m\times v} & 0_{m\times m} \end{bmatrix},$$

where $m = n - v$. The public key matrices are $P_i = S^\top F_i S$ for $i \in [m]$. The essence of UOV lies in its linearity with respect to the oil variables, where the lower-right block of F_i is zero, enabling the system to be solved once the vinegar variables are fixed.

QR-UOV is constructed with a matrix representation of quotient ring elements. Let ℓ be a positive integer and $f \in \mathbb{F}_q[x]$ be an irreducible polynomial with $\deg f = \ell$. For any $g \in \mathbb{F}_q[x]/(f)$, we define an $\ell \times \ell$ *polynomial matrix* $\Phi_g^f \in \mathbb{F}_q^{\ell\times\ell}$ such that

$$(1, x, \ldots, x^{\ell-1})\, \Phi_g^f = (g, xg, \ldots, x^{\ell-1}g) \pmod f.$$

This induces an injective ring homomorphism $g \mapsto \Phi_g^f : \mathbb{F}_q[x]/(f) \hookrightarrow \mathbb{F}_q^{\ell\times\ell}$; namely, $\Phi_{g_1+g_2}^f = \Phi_{g_1}^f + \Phi_{g_2}^f$ and $\Phi_{g_1 g_2}^f = \Phi_{g_1}^f \Phi_{g_2}^f$ hold for any pair of g_1 and g_2. Since each matrix Φ_g^f is determined by the ℓ coefficients of g, the public key can be represented compactly using only ℓ elements of $\mathbb{F}_q$. This enables a compact and efficient public key representation, achieving both compression and structural efficiency.

2.2 QR-UOV algorithm

In this section, we follow Round 2 specification of QR-UOV [AFI+24]. The algorithms for key generation, signature generation, and verification are given in Algorithms 1, 2, and 3, respectively. Notations basically follow the specification.

Key generation. Algorithm 1 describes the key generation in compressed form, which uses two seeds. Let $f \in \mathbb{F}_q[x]$ be an ℓ-degree irreducible polynomial defining $\mathbb{F}_{q^\ell} = \mathbb{F}_q[x]/(f)$, and $\Phi_g^f \in \mathbb{F}_q^{\ell\times\ell}$ denote the *polynomial matrix* associated with $g \in \mathbb{F}_{q^\ell}$. We define $\mathcal{A}_f = \{\Phi_g^f \mid g \in \mathbb{F}_{q^\ell}\}$ and let $W \in \mathbb{F}_q^{\ell\times\ell}$ be a symmetrizer matrix such that $W\Phi_g^f$ is symmetric for all g. We write $W^{(a)}$ for the block-diagonal matrix with a copies of W on the diagonal, i.e., $W^{(a)} = \mathrm{diag}(W, \ldots, W) \in \mathbb{F}_q^{a\ell\times a\ell}$. Let $N \coloneqq V + M$ with $v = \ell V$, $m = \ell M$; hence $n = \ell N = v + m$. Here $\mathcal{A}_f^{a\times b}$ denotes the set of $a \times b$ block matrices whose blocks lie in $\mathcal{A}_f$ (each block is $\ell \times \ell$). Then the public key matrices P_i ($i \in [m]$) and the central

Algorithm 1 KeyGen()

Output: public key $\mathsf{pk} \in \{0,1\}^\lambda \times \left(\mathbb{F}_{q^\ell}^{M\times M}\right)^m$ and private key $\mathsf{sk} \in \{0,1\}^{2\lambda}$

1: $(\mathsf{seed}_{\mathsf{pk}}, \mathsf{seed}_{\mathsf{sk}}) \xleftarrow{\$} \{0,1\}^{2\lambda}$ ▷ $\mathsf{seed}_{\mathsf{pk}}, \mathsf{seed}_{\mathsf{sk}} \in \{0,1\}^\lambda$
2: $\bar{S}' \leftarrow \mathsf{Expand}_{\mathsf{sk}}(\mathsf{seed}_{\mathsf{sk}})$ ▷ $\bar{S}' \in \mathbb{F}_{q^\ell}^{V\times M}$
3: **for** i from 1 to m **do**
4: $\quad(\bar{P}_{i,1}, \bar{P}_{i,2}) \leftarrow \mathsf{Expand}_{\mathsf{pk}}(\mathsf{seed}_{\mathsf{pk}}, i)$ ▷ $\bar{P}_{i,1} \in \mathbb{F}_{q^\ell}^{V\times V}$ (symmetric), $\bar{P}_{i,2} \in \mathbb{F}_{q^\ell}^{V\times M}$
5: $\quad\bar{P}_{i,3} \leftarrow -\bar{S}'^\top \bar{P}_{i,1}\bar{S}' + \bar{P}_{i,2}^\top \bar{S}' + \bar{S}'^\top \bar{P}_{i,2}$ ▷ $\bar{P}_{i,3} \in \mathbb{F}_{q^\ell}^{M\times M}$ (symmetric)
6: **end for**
7: **return** $(\mathsf{pk}, \mathsf{sk}) = ((\mathsf{seed}_{\mathsf{pk}}, \{\bar{P}_{i,3}\}_{i\in[m]}), (\mathsf{seed}_{\mathsf{sk}}, \mathsf{seed}_{\mathsf{pk}}))$

map matrices F_i belong to $W^{(N)}\mathcal{A}_f^{N\times N}$, while the private linear map matrices S belong to $\mathcal{A}_f^{N\times N}$. Since each block is $\ell \times \ell$, we have $P_i, F_i, S \in \mathbb{F}_q^{n\times n}$.

For each $i \in [m]$, we write P_i, F_i, and S in block form as

$$P_i = \begin{pmatrix} P_{i,1} & P_{i,2} \\ P_{i,2}^\top & P_{i,3} \end{pmatrix}, \quad F_i = \begin{pmatrix} F_{i,1} & F_{i,2} \\ F_{i,2}^\top & 0_{m\times m} \end{pmatrix}, \quad S = \begin{pmatrix} I_v & S' \\ 0_{m\times v} & I_m \end{pmatrix}.$$

Here $P_{i,1}, F_{i,1} \in W^{(V)}\mathcal{A}_f^{V\times V}$, $P_{i,2}, F_{i,2} \in W^{(V)}\mathcal{A}_f^{V\times M}$, $P_{i,3} \in W^{(M)}\mathcal{A}_f^{M\times M}$, and $S' \in \mathcal{A}_f^{V\times M}$. We use the fundamental relation $P_i = S^\top F_i S$, which expands to

$$\begin{aligned} F_{i,1} &= P_{i,1}, \\ F_{i,2} &= -P_{i,1}S' + P_{i,2}, \\ P_{i,3} &= -S'^\top P_{i,1} S' + P_{i,2}^\top S' + S'^\top P_{i,2}. \end{aligned}$$

Hence $P_{i,3}$ is determined by $P_{i,1}, P_{i,2}$, and S', which allows seed-based key compression of Czypek et al. [CHT12]. In QR-UOV, the key pair is represented in compressed form as $\mathsf{pk} = (\mathsf{seed}_{\mathsf{pk}}, \{P_{i,3}\}_{i\in[m]})$ and $\mathsf{sk} = (\mathsf{seed}_{\mathsf{sk}}, \mathsf{seed}_{\mathsf{pk}})$. In this representation, $\mathsf{seed}_{\mathsf{pk}}$ expands to pseudorandom blocks $\{P_{i,1}, P_{i,2}\}_{i\in[m]}$, whereas $\mathsf{seed}_{\mathsf{sk}}$ expands to S'.

For efficient implementation, the key generation works not directly with P_i, F_i, S, but with their counterparts $\bar{P}_i, \bar{F}_i, \bar{S}$ over the extension field $\mathbb{F}_{q^\ell}$. This is achieved via the *pull-back method* [FIKT21]. We define

$$\phi : \mathcal{A}_f^{a,b} \to \mathbb{F}_{q^\ell}^{a\times b}, \quad \left(\Phi_{g_{i,j}}^f\right)_{i,j} \mapsto (g_{i,j})_{i,j},$$

and write $\bar{P}_i = \phi\left((W^{(N)})^{-1}P_i\right)$, $\bar{F}_i = \phi\left((W^{(N)})^{-1}F_i\right)$, and $\bar{S} = \phi(S)$. Then the relation $P_i = S^\top F_i S$ corresponds to $\bar{P}_i = \bar{S}^\top \bar{F}_i \bar{S}$ over $\mathbb{F}_{q^\ell}$.

Algorithm 1 implements the compressed key generation described above over $\mathbb{F}_{q^\ell}$: it expands $\mathsf{seed}_{\mathsf{pk}}$ into $\bar{P}_{i,1}, \bar{P}_{i,2}$ and $\mathsf{seed}_{\mathsf{sk}}$ into $\bar{S}'$, and then computes

$$\bar{P}_{i,3} = -\bar{S}'^\top \bar{P}_{i,1}\bar{S}' + \bar{P}_{i,2}^\top \bar{S}' + \bar{S}'^\top \bar{P}_{i,2},$$

which is exactly the barred version of the relation for $P_{i,3}$.

Signature generation. Algorithm 2 describes the signature generation, which takes a message $\boldsymbol{M}$ and a private key sk as input and produces a signature σ. The procedure basically follows UOV with modifications to ensure EUF–CMA security.

First, choose vinegar variables $\boldsymbol{y}_V = (y_1, \ldots, y_v)^\top \in \mathbb{F}_q^v$ uniformly at random. Generate a λ-bit salt r, compute $\mu = \mathsf{Hash}(\mathsf{seed}_{\mathsf{pk}} \| \boldsymbol{M})$, and set $\boldsymbol{t} := \mathsf{Hash}(\mu, r) \in \mathbb{F}_q^m$. Let the oil variables be $\boldsymbol{y}_O = (y_{v+1}, \ldots, y_n)^\top \in \mathbb{F}_q^m$. For each $i \in [m]$, the central equation given as

$$(\boldsymbol{y}_V^\top, \boldsymbol{y}_O^\top) \begin{pmatrix} F_{i,1} & F_{i,2} \\ F_{i,2}^\top & 0_{m\times m} \end{pmatrix} \begin{pmatrix} \boldsymbol{y}_V \\ \boldsymbol{y}_O \end{pmatrix} = t_i,$$

Algorithm 2 $\mathsf{Sign}(\boldsymbol{M}, \mathsf{sk})$

Input: message $\boldsymbol{M} \in \mathbb{B}^*$ and private key $\mathsf{sk} \in \{0,1\}^{2\lambda}$
Output: signature $\sigma \in \{0,1\}^{\lambda} \times \mathbb{F}_q^m$

$(\mathsf{seed}_{\mathsf{sk}}, \mathsf{seed}_{\mathsf{pk}}) \xleftarrow{\$} \mathsf{sk}$
$\bar{S}' \leftarrow \mathsf{Expand}_{\mathsf{sk}}(\mathsf{seed}_{\mathsf{sk}})$
$\boldsymbol{y} = (y_1, \ldots, y_v)^\top \xleftarrow{\$} \mathbb{F}_q^v$
for i from 1 to m **do**
  $(\bar{P}_{i,1}, \bar{P}_{i,2}) \leftarrow \mathsf{Expand}_{\mathsf{pk}}(\mathsf{seed}_{\mathsf{pk}}, i)$
  $\bar{\boldsymbol{y}} \leftarrow W^{(V)}\phi^{-1}(\bar{P}_{i,1})\boldsymbol{y}$ ▷ $\bar{\boldsymbol{y}} \in \mathbb{F}_q^v$
  $\boldsymbol{L}_i \leftarrow -2\phi^{-1}(\bar{S}')^\top \bar{\boldsymbol{y}} + 2W^{(M)}\phi^{-1}(\bar{P}_{i,2}^\top)\boldsymbol{y}$ ▷ $\boldsymbol{L}_i \in \mathbb{F}_q^m$
  $u_i \leftarrow \boldsymbol{y}^\top \bar{\boldsymbol{y}}$ ▷ $u_i \in \mathbb{F}_q$
end for
$L \leftarrow (\boldsymbol{L}_1, \ldots, \boldsymbol{L}_m)^\top$ ▷ $L \in \mathbb{F}_q^{m \times m}$
$\boldsymbol{u} \leftarrow (u_1, \ldots, u_m)^\top$ ▷ $\boldsymbol{u} \in \mathbb{F}_q^m$
$\mu \leftarrow \mathsf{SHAKE256}(\mathsf{seed}_{\mathsf{pk}} || \mathsf{BytesToBits}(\boldsymbol{M}), 512)$
repeat
  $r \xleftarrow{\$} \{0,1\}^{\lambda}$
  $\boldsymbol{t} \leftarrow \mathsf{Hash}(\mu, r)$ ▷ $\boldsymbol{t} \in \mathbb{F}_q^m$
until $L\boldsymbol{x} = \boldsymbol{t} - \boldsymbol{u}$ has solutions for $\boldsymbol{x}$.
Choose one solution $(y_{v+1}, \ldots, y_n) \in \mathbb{F}_q^m$ of $L\boldsymbol{x} = \boldsymbol{t} - \boldsymbol{u}$ randomly.
$\boldsymbol{s} \leftarrow (y_1, \ldots, y_v, y_{v+1}, \ldots, y_n)^\top - \left(\phi^{-1}(\bar{S}') \cdot (y_{v+1}, \ldots, y_n)^\top || \boldsymbol{0}_m\right)$ ▷ $\boldsymbol{s} \in \mathbb{F}_q^n$
return $\sigma = (r, \boldsymbol{s})$

is linear in $\boldsymbol{y}_O$ and is equivalent to

$$2\boldsymbol{y}_V^\top F_{i,2}\boldsymbol{y}_O = t_i - \boldsymbol{y}_V^\top F_{i,1}\boldsymbol{y}_V.$$

Stacking these m equations yields a linear system of

$$L(\boldsymbol{y}_V)\boldsymbol{y}_O = \boldsymbol{t} - u(\boldsymbol{y}_V),$$

where the i-th row of $L(\boldsymbol{y}_V) \in \mathbb{F}_q^{m \times m}$ is $2\boldsymbol{y}_V^\top F_{i,2}$ and the i-th entry of $u(\boldsymbol{y}_V) \in \mathbb{F}_q^m$ is $\boldsymbol{y}_V^\top F_{i,1}\boldsymbol{y}_V$.

If the system has at least one solution, choose one $\boldsymbol{y}_O$ uniformly at random from the solution set, and set $\boldsymbol{s} = \mathcal{S}^{-1}(\boldsymbol{y}_V; \boldsymbol{y}_O)$. Otherwise, resample the salt r and retry; QR-UOV permits singular $L(\boldsymbol{y}_V)$. In practice, the rank deficiency is small, so a random $\boldsymbol{t}$ is solvable with an overwhelming probability. Finally, it outputs the signature $\sigma = (r, \boldsymbol{s})$.

Algorithm 2 expands $F_{i,2}$ via $F_{i,2} = -P_{i,1}S' + P_{i,2}$ and applies matrix-chain optimization (Section 3.2). Its variable $\boldsymbol{y}$ equals the vinegar variables $\boldsymbol{y}_V$ in the text.

Signature verification. Algorithm 3 describes the verification, which takes a message $\boldsymbol{M}$, a public key pk, and a signature σ as input, and outputs either accept or reject. The verification procedure follows the same steps as in UOV. First, a value $\boldsymbol{t} \in \mathbb{F}_q^m$ is computed as $\boldsymbol{t} = \mathsf{Hash}(\mu, r)$, where μ is obtained by hashing the public seed and the message, $\mu = \mathsf{Hash}(\mathsf{seed}_{\mathsf{pk}} \| \boldsymbol{M})$. Next, the signature $\boldsymbol{s} \in \mathbb{F}_q^n$ is substituted into the public key map $\mathcal{P}$ to compute $\boldsymbol{t}' \in \mathbb{F}_q^m$, i.e., $\boldsymbol{t}' = \mathcal{P}(\boldsymbol{s})$. The signature σ is accepted if and only if $\boldsymbol{t} = \boldsymbol{t}'$; otherwise rejected.

Seed expansion. In QR-UOV, parts of the public key, $\bar{P}_{i,1}$ and $\bar{P}_{i,2}$, as well as the private key, $\bar{S}'$, are derived from short seeds using a pseudorandom generator (PRG), namely

Algorithm 3 $\mathsf{Verify}(\boldsymbol{M}, \mathsf{pk}, \sigma)$

Input: message $\boldsymbol{M} \in \mathbb{B}^*$, public key $\mathsf{pk} \in \{0,1\}^\lambda \times \left(\mathbb{F}_{q^\ell}^{M \times M}\right)^m$, and signature $\sigma \in \{0,1\}^\lambda \times \mathbb{F}_q^m$
Output: accept or reject

1: $(\mathsf{seed}_{\mathsf{pk}}, \{\bar{P}_{i,3}\}_{i\in[m]}) \leftarrow \mathsf{pk}$
2: $(r, \boldsymbol{s}) \leftarrow \sigma$
3: **for** i from 1 to m **do**
4: $\quad (\bar{P}_{i,1}, \bar{P}_{i,2}) \leftarrow \mathsf{Expand}_{\mathsf{pk}}(\mathsf{seed}_{\mathsf{pk}}, i)$
5: $\quad P_i \leftarrow W^{(N)}\phi^{-1}\begin{pmatrix} \bar{P}_{i,1} & \bar{P}_{i,2} \\ \bar{P}_{i,2}^\top & \bar{P}_{i,3} \end{pmatrix}$ ▷ $P_i \in \mathbb{F}_q^{n\times n}$
6: **end for**
7: $\mu \leftarrow \mathsf{SHAKE256}(\mathsf{seed}_{\mathsf{pk}} || \mathsf{BytesToBits}(\boldsymbol{M}), 512)$
8: $\boldsymbol{t} \leftarrow \mathsf{Hash}(\mu, r)$
9: $\boldsymbol{t'} \leftarrow (\boldsymbol{s}^\top P_1 \boldsymbol{s}, \ldots, \boldsymbol{s}^\top P_m \boldsymbol{s})^\top$ ▷ $\boldsymbol{t'} \in \mathbb{F}_q^m$
10: **return** accept if $\boldsymbol{t} = \boldsymbol{t'}$ and reject otherwise.

AES–CTR[1]. The generation of $\bar{P}_{i,1}$ and $\bar{P}_{i,2}$ uses independent PRG streams to enable parallel execution. The PRG output is mapped to uniformly random elements in $\mathbb{F}_q$ via rejection sampling (RejSamp), and these elements are then arranged to form a $\mathbb{F}_{q^\ell}$ matrix.

RejSamp operates in a bytewise manner. To generate n' elements in $\mathbb{F}_q$, we read a PRG byte string of fixed length τ as defined in the QR-UOV specification ($\tau > n'$). Each byte is then bitwise-AND masked with q (recall $q = 2^k - 1$), producing values in $\{0, \ldots, q\}$. We partitioned the masked sequence into two parts: the head of n' bytes and the tail of $\tau - n'$ bytes. In the head, every occurrence of the symbol q is replaced by a non-q value taken from the tail. This yields n' valid elements in $\mathbb{F}_q$. The specification sets τ large enough that the tail contains sufficient non-q values for all replacements, so no additional bytes are required.

In the $\mathsf{Expand}_{\mathsf{sk}}(\mathsf{seed}_{\mathsf{sk}})$ algorithm, the private key matrix $\bar{S}' \in \mathbb{F}_{q^\ell}^{V\times M}$ is expanded by generating ℓVM elements over $\mathbb{F}_q$ using RejSamp, followed by constructing polynomials in row-major order, forming each entry in the reverse-degree order. Similarly, in $\mathsf{Expand}_{\mathsf{pk}}(\mathsf{seed}_{\mathsf{pk}}, i)$, the symmetric public key matrix $\bar{P}_{i,1} \in \mathbb{F}_{q^\ell}^{V\times V}$ is generated by sampling only the upper-triangular (including the diagonal) part, producing $\ell V(V+1)/2$ elements in $\mathbb{F}_q$ via RejSamp and mirroring to the lower triangle. $\bar{P}_{i,2} \in \mathbb{F}_{q^\ell}^{V\times M}$ is filled analogously to $\bar{S}'$ with ℓVM samples.

Parameters. QR-UOV targets security levels I, III, and V, with base-field sizes $q = 7, 31, 127$, respectively. Table 1 lists the tuple (q, v, m, ℓ) for each candidate set; among these, the configuration with $q = 127$ and $\ell = 3$ is recommended for efficiency. Table 2 records the auxiliary parameters (q, ℓ, f, W), where $f \in \mathbb{F}_q[x]$ is an irreducible polynomial of degree ℓ and $W \in \mathbb{F}_q^{\ell\times\ell}$ is the symmetrizer matrix. Since W is expressed using J_i in the table, we write J_i for the $i \times i$ anti-identity matrix (e.g., $J_2 = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$). The security parameter λ, which fixes the sizes of $\mathsf{seed}_{\mathsf{pk}}$, $\mathsf{seed}_{\mathsf{sk}}$, and the salt r, is set to 128, 192, and 256 for levels I, III, and V, respectively.

2.3 Field-matrix correspondence for QR-UOV computations

In the QR-UOV specification, an element $g \in \mathbb{F}_{q^\ell}$ is represented over the base field $\mathbb{F}_q$ by the *polynomial matrix* Φ_g^f, and subsequent computations are carried out as matrix

[1]In the QR-UOV Round 2 specification, there are two options for PRG: SHAKE and AES–CTR. In this paper, we evaluate the faster AES-CTR variant.

Table 1: Parameters for QR-UOV

SL	q	v	m	ℓ
I	127	156	54	3
	31	165	60	3
	31	600	70	10
	7	740	100	10
III	127	228	78	3
	31	246	87	3
	31	890	100	10
	7	1100	140	10
V	127	306	105	3
	31	324	114	3
	31	1120	120	10
	7	1490	190	10

Table 2: Auxiliary parameters for QR-UOV

(q, ℓ)	f	W
$(127, 3)$	$x^3 - x - 1$	$\begin{pmatrix} J_1 & \\ & J_2 \end{pmatrix}$
$(31, 3)$	$x^3 - x - 1$	$\begin{pmatrix} J_1 & \\ & J_2 \end{pmatrix}$
$(31, 10)$	$x^{10} - 2x - 1$	$\begin{pmatrix} J_1 & \\ & J_9 \end{pmatrix}$
$(7, 10)$	$x^{10} - 5x^3 - 1$	$\begin{pmatrix} J_3 & \\ & J_7 \end{pmatrix}$

operations over $\mathbb{F}_q$. In contrast, the QR-UOV reference implementation [FIH+23, AFI+24] performs equivalent computations directly over $\mathbb{F}_{q^\ell}$ without explicitly converting to a matrix representation, to reduce memory traffic and improve cache locality. We describe this correspondence here, using the symmetrizer matrix W defined in Section 2.2 (i.e., $(\Phi_g^f)^\top W = W\Phi_g^f$ for all g).

Let $a \in \mathbb{F}_{q^\ell} = a = a_0 + a_1 x + \cdots + a_{\ell-1}x^{\ell-1}$. Define

$$\mathsf{Coef}(a) \coloneqq (a_0, \ldots, a_{\ell-1})^\top \in \mathbb{F}_q^\ell.$$

For a vector $\boldsymbol{u} = (u_1, \ldots, u_t) \in \mathbb{F}_{q^\ell}^t$, also define

$$\mathsf{Coef}(\boldsymbol{u}) \coloneqq \mathsf{Coef}(u_1) \,\|\, \cdots \,\|\, \mathsf{Coef}(u_t) \in \mathbb{F}_q^{t\ell}.$$

By the definition, for all $g, h \in \mathbb{F}_{q^\ell}$,

$$\Phi_g^f \,\mathsf{Coef}(h) = \mathsf{Coef}(gh),$$

which identifies multiplication by g in $\mathbb{F}_{q^\ell}$ with left multiplication by Φ_g^f on coefficient vectors.

Let $e_0 \coloneqq (1, 0, \ldots, 0)^\top \in \mathbb{F}_q^\ell$ and define the W-extraction

$$\mathsf{Extr}_W(\gamma) \coloneqq \mathsf{Coef}(\gamma)^\top W e_0 \in \mathbb{F}_q \qquad (\gamma \in \mathbb{F}_{q^\ell}).$$

Here, the following identities hold:

$$\begin{aligned} \mathsf{Extr}_W(\alpha\beta) &= \mathsf{Coef}(\alpha)^\top W\, \mathsf{Coef}(\beta) && \text{for any } \alpha, \beta \in \mathbb{F}_{q^\ell}, \\ \mathsf{Extr}_W\big(\boldsymbol{u}^\top A\, \boldsymbol{w}\big) &= \mathsf{Coef}(\boldsymbol{u})^\top W^{(t)}\, \mathsf{Coef}(A\boldsymbol{w}) && \text{for any } t \geq 1,\ \boldsymbol{u}, \boldsymbol{w} \in \mathbb{F}_{q^\ell}^t,\ A \in \mathbb{F}_{q^\ell}^{t\times t}. \end{aligned}$$

For the scalar case, it is proven as

$$\begin{aligned} \mathsf{Extr}_W(\alpha\beta) &= \mathsf{Coef}(\alpha\beta)^\top W e_0 = \mathsf{Coef}(\beta)^\top (\Phi_\alpha^f)^\top W e_0 \\ &= \mathsf{Coef}(\beta)^\top W\, \Phi_\alpha^f e_0 = \mathsf{Coef}(\beta)^\top W\, \mathsf{Coef}(\alpha). \end{aligned}$$

Table 3: Correspondence between $\mathbb{F}_q$-matrix formulas and $\mathbb{F}_{q^\ell}$-level evaluation

Function	$\mathbb{F}_q$ formula (Specification)	$\mathbb{F}_{q^\ell}$ formula (Implementation)
L in Sign	$2\,\boldsymbol{y}^\top W^{(V)}\phi^{-1}(\bar{F}_{i,2})$	$W^{(M)}\mathsf{Coef}\big(2\,\tilde{\boldsymbol{y}}^\top \bar{F}_{i,2}\big)$
$\boldsymbol{u}$ in Sign	$\boldsymbol{y}^\top W^{(V)}\phi^{-1}(\bar{F}_{i,1})\boldsymbol{y}$	$\mathsf{Extr}_W(\tilde{\boldsymbol{y}}^\top \bar{F}_{i,1}\tilde{\boldsymbol{y}})$
$\boldsymbol{t}'$ in Verify	$\boldsymbol{s}^\top W^{(N)}\phi^{-1}(\bar{P}_i)\boldsymbol{s}$	$\mathsf{Extr}_W(\tilde{\boldsymbol{s}}^\top \bar{P}_i\tilde{\boldsymbol{s}})$

The vector case follows by expanding $\boldsymbol{u}^\top A\boldsymbol{w} = \sum_i u_i(A\boldsymbol{w})_i$ and stacking the scalar identity across i. Given $t \in \mathbb{N}$ and $\boldsymbol{s} \in \mathbb{F}_q^{t\ell}$, we write $\tilde{\boldsymbol{s}} \in \mathbb{F}_{q^\ell}^{t}$ for the unique vector whose base-field coefficient vector equals $\boldsymbol{s}$, i.e., $\mathsf{Coef}(\tilde{\boldsymbol{s}}) = \boldsymbol{s}$. With this notation, the correspondence between the specification's $\mathbb{F}_q$-matrix formulas and the reference implementation's $\mathbb{F}_{q^\ell}$-level evaluation is summarized in Table 3. The correspondence in Table 3 forms the foundation for our subsequent algorithm-level optimizations described in Section 3.

In the QR-UOV parameter sets, W is a blockwise permutation matrix (a direct sum of anti-identity blocks), so $W\,\mathsf{Coef}(\cdot)$ is a fixed permutation of coefficients (blockwise reversal). Equivalently, for $a \in \mathbb{F}_{q^\ell}$ with $\boldsymbol{u} \coloneqq \mathsf{Coef}(a)$, $\mathsf{Extr}_W(a) = \mathsf{Coef}(a)^\top We_0$ picks the entry of $\boldsymbol{u}$ at the unique coordinate where We_0 has value 1.[2]

3 Algorithm-level acceleration methods

This section presents algorithm-level acceleration methods, such as reordering arithmetic operations. First, we describe a method for applying the Toom–Cook-like algorithm, named Evaluation–MMFQ–Interpolation (EMI) method, to matrix multiplication in the key generation to achieve efficient polynomial multiplication. Next, we show a method for reducing the computational cost by changing the order of matrix multiplication, thereby converting it into a vector-matrix multiplication in the signature generation. Furthermore, we describe a method that uses a single inner product with precomputed coefficients in the signature verification.

3.1 Evaluation–MMFQ–Interpolation (EMI) method for $\mathbb{F}_{q^\ell}$ matrix multiplication

The proposed EMI method computes matrix products over $\mathbb{F}_{q^\ell}$ in three steps. First, it evaluates the entries over $\mathbb{F}_q$; next, it performs n_{eval} matrix-matrix multiplications over $\mathbb{F}_q$ (MMFQ); and finally, it interpolates entrywise back to $\mathbb{F}_{q^\ell}$. Let $G \in \mathbb{F}_q^{n_{\mathrm{eval}}\times\ell}$ and $U \in \mathbb{F}_q^{\ell\times n_{\mathrm{eval}}}$ be the evaluation and interpolation matrices, respectively. They depend on the parameter set and the chosen evaluation points. For simplicity, we describe the case of multiplication in $\mathbb{F}_{q^\ell}$ first. Let $a, b \in \mathbb{F}_{q^\ell}$ and let $\boldsymbol{a}$ and $\boldsymbol{b}$ be the corresponding ℓ-dimensional coefficient vectors over $\mathbb{F}_q$. The coefficient vector of the product ab is computed by the following three steps: (i) forming $\boldsymbol{v}_a = G\boldsymbol{a}$ and $\boldsymbol{v}_b = G\boldsymbol{b} \in \mathbb{F}_q^{n_{\mathrm{eval}}}$, (ii) multiplying them element-wise (i.e., Hadamard product), and (iii) mapping back via $\boldsymbol{c} = U(\boldsymbol{v}_a \odot \boldsymbol{v}_b)$, where $\odot$ is the Hadamard product operator. Note here that since the reduction modulo the irreducible polynomial is integrated into U, no separate reduction step is needed.

The matrix multiplication is performed by the above $\mathbb{F}_{q^\ell}$ multiplications multiple times. Given $A \in \mathbb{F}_{q^\ell}^{X\times Y}$ and $B \in \mathbb{F}_{q^\ell}^{Y\times Z}$, we compute the matrix product $C = AB$ in the above manner. We first evaluate every element of A and B using G. For each evaluation index

[2] In the QR-UOV reference code, the permutation induced by W is implemented by the macro `QRUOV_perm`. Specifically, $\boldsymbol{w} \coloneqq W\,\boldsymbol{u}$ is implemented by `w[QRUOV_perm(i)]=u[i]`, and $\mathsf{Extr}_W(a) = \mathsf{Coef}(a)^\top We_0$ is implemented by `u[QRUOV_perm(0)]`.

Table 4: Number of $\mathbb{F}_q$ multiplications in KeyGen ($\times 10^9$)

SL	(q, v, m, ℓ)	Schoolbook	EMI method	Ratio
I	(127, 156, 54, 3)	40	26	0.67
	(31, 165, 60, 3)	56	37	0.66
	(31, 600, 70, 10)	217	106	0.49
	(7, 740, 100, 10)	695	342	0.49
III	(127, 228, 78, 3)	177	112	0.64
	(31, 246, 87, 3)	260	163	0.63
	(31, 890, 100, 10)	970	387	0.40
	(7, 1150, 140, 10)	3223	1337	0.41
V	(127, 306, 105, 3)	580	357	0.62
	(31, 324, 114, 3)	774	473	0.61
	(31, 1120, 120, 10)	2193	798	0.36
	(7, 1490, 190, 10)	10058	3691	0.37

j ($1 \le j \le n_{\mathrm{eval}}$), we obtain $\mathbb{F}_q$-matrices $A^{(j)} \in \mathbb{F}_q^{X\times Y}$ and $B^{(j)} \in \mathbb{F}_q^{Y\times Z}$, collecting the j-th value from the evaluation result. We then compute matrix multiplications over $\mathbb{F}_q$ $C^{(j)} = A^{(j)}B^{(j)} \in \mathbb{F}_q^{X\times Z}$ for all j. We finally perform the interpolation by reversing $C^{(j)}$ into column vectors over $\mathbb{F}_q$ and applying U to them. In summary, an $\mathbb{F}_{q^\ell}$-matrix product is carried out by two fixed $\mathbb{F}_q$-linear passes (entrywise application of G and U) around n_{eval} independent matrix multiplications over $\mathbb{F}_q$.

The choice of n_{eval} and the construction of (G, U) follow two standard sources of evaluation points. $n_{\mathrm{eval}} = 2\ell - 1$ in the polynomial (Toom-type) case. Practically, we evaluate $2\ell - 2$ distinct finite points in $\mathbb{F}_q$ and also record the highest-degree coefficient of the polynomial product, which corresponds to the value at ∞. This is possible whenever $q+1 \ge 2\ell-1$, since $\mathbb{F}_q$ contributes to q finite points and the highest-degree coefficient adds one more entry. With the QR-UOV parameter set, we have $n_{\mathrm{eval}} = 5$ for $(q, \ell) = (31, 3)$ and $(127, 3)$. For $(q, \ell) = (31, 10)$, we have $n_{\mathrm{eval}} = 19$. The corresponding evaluation matrix G is a truncated Vandermonde (one row per finite point) with an additional selector row $(0, \ldots, 0, 1)$ that extracts the highest-degree coefficient. The interpolation matrix U is the product SL, where L performs Lagrange interpolation from $2\ell - 1$ values to the coefficient vector, and S reduces modulo the irreducible polynomial to obtain an ℓ-vector.

In contrast, if $q + 1 < 2\ell - 1$, we use the Chudnovsky–Chudnovsky algorithm [CC88, Ran12, BPR+21]. The evaluation points come from a function space on a suitable algebraic curve. We use the same matrix format $G \in \mathbb{F}_q^{n_{\mathrm{eval}}\times\ell}$ and $U \in \mathbb{F}_q^{\ell\times n_{\mathrm{eval}}}$ to describe the computation. For the QR-UOV parameter $(q, \ell) = (7, 10)$, a genus-4 curve with 24 $\mathbb{F}_7$-rational points gives $n_{\mathrm{eval}} = 23$. The matrices (G, U) in our implementation of this case are given in Appendix A.

Considering the multiplication of two $N \times N$ matrices over $\mathbb{F}_{q^\ell}$, we estimate the computational cost of the EMI method. The schoolbook method requires $O(\ell^2 N^3)$ multiplications in $\mathbb{F}_q$. In contrast, the EMI method consists of three phases: evaluation, matrix multiplication, and interpolation. Since every entry is transformed by matrices G and U, the evaluation and interpolation phases incur the costs of $O(n_{\mathrm{eval}}\ell N^2)$, respectively. The matrix-multiplication phase incurs a cost of $O(n_{\mathrm{eval}}N^3)$ multiplications in total. For sufficiently large N, the cubic term dominates; thus, replacing the ℓ^2 factor with n_{eval} in the dominant term renders the EMI method highly effective. Table 4 reports the $\mathbb{F}_q$-multiplication counts per KeyGen for the schoolbook and EMI methods. Here, the total numbers of $\mathbb{F}_q$-multiplication decreases by approximately 30% and 50% for $\ell = 3$ and $\ell = 10$, respectively. Since we did not exploit the sparsity of G and U for estimation

in Table 4, the total cost of the EMI method could be further reduced in optimized implementations.

3.2 Matrix chain multiplication for Sign

QR-UOV's Sign algorithm initially computes the central map $F_{i,2}$. In fact, the signature generation only requires the product $\boldsymbol{y}^\top F_{i,2}$ rather than $F_{i,2}$ itself. In the following, we present a method that computes the signature without explicitly forming $F_{i,2}$.

For the linear system $L\boldsymbol{x} = \boldsymbol{t} - \boldsymbol{u}$, expanding $F_{i,2} = -P_{i,1}S' + P_{i,2}$ yields

$$\boldsymbol{L}_i^\top = 2\boldsymbol{y}^\top F_{i,2} = 2\boldsymbol{y}^\top(-P_{i,1}S' + P_{i,2}) = -2\boldsymbol{y}^\top P_{i,1}S' + 2\boldsymbol{y}^\top P_{i,2}.$$

The first term can be calculated by performing two vector-matrix multiplications in order from the left. This reduces the calculation cost by approximately 90% compared to that of the matrix-matrix multiplication $P_{i,1}S'$, resulting in a significant speedup.

This method was adopted in the QR-UOV Round 2 specification [AFI+24] (Algorithm 2). This method is also applicable to the pkc+skc variant of the UOV Round 2 proposal [BCD+24] with key compression.

This method is designed for scenarios in which the key is expanded once per signature. Conversely, if multiple signatures are generated under the same key, a different strategy becomes preferable: expand the key once, and precompute (as well as cache) $F_{i,2}$.

3.3 QR-UOV public map evaluation

The main operation in signature verification is the calculation of the quadratic form $\boldsymbol{s}^\top P_i \boldsymbol{s}$, which substitutes the solution $\boldsymbol{s}$ into the m equations represented by the public key P_i ($i \in [m]$). Here, the calculation can be written as follows:

$$\boldsymbol{s}^\top P_i \boldsymbol{s} = \sum_{j,k} s_j s_k P_{i,j,k}.$$

According to [CKY21], a typical UOV implementation evaluates the quadratic form as follows: for each pair (j,k) with $j \le k$, it first computes the pairwise product $s_j s_k$, multiplies it by the public-key coefficients $P_{i,j,k}$ for all $i \in [m]$, and accumulates the results into w_i. To efficiently execute the above, the public key is arranged as a Macaulay matrix of size $\mathbb{F}_q^{(n(n+1)/2)\times m}$, where each row indexed by (j,k) stores the contiguous block $(P_{1,j,k}, \ldots, P_{m,j,k})$ in memory.

In QR-UOV, since the public key P is compressed using $\mathbb{F}_{q^\ell}$ and identities in Section 2.3, we can write the public map evaluation as follows:

$$\begin{aligned}
\boldsymbol{s}^\top P_i \boldsymbol{s} &= \mathsf{Extr}_W(\tilde{\boldsymbol{s}}^\top \bar{P}_i \tilde{\boldsymbol{s}}) \\
&= \mathsf{Extr}_W\left(\sum_{j,k} \tilde{s}_j \tilde{s}_k \bar{P}_{i,j,k}\right) \\
&= \sum_{j,k} \mathsf{Extr}_W(\tilde{s}_j \tilde{s}_k \bar{P}_{i,j,k}) \\
&= \sum_{j,k} \mathsf{Coef}(\bar{P}_{i,j,k})^\top W\, \mathsf{Coef}(\tilde{s}_j \tilde{s}_k) \\
&= \mathsf{Coef}(\bar{P}_i) \cdot \boldsymbol{u}.
\end{aligned}$$

More specifically, after creating the vector $\boldsymbol{u} \in \mathbb{F}_q$ from $\boldsymbol{s}$, we calculate $\boldsymbol{s}^\top P_i \boldsymbol{s}$ with the inner product of $\boldsymbol{u}$ and the coefficient vector of $\bar{P}_i$. For this purpose, we first create a

Table 5: Typical x86 integer multiplication instructions

Instruction	Extension	Operation
`PMULLW`	SSE2	`PMULLW`$(\boldsymbol{a}, \boldsymbol{b}) = (a_0 b_0, \dots)$. Multiply 16-bit signed integers and store lower 16 bits of 32-bit product.
`PMULHUW`	SSE2	`PMULHUW`$(\boldsymbol{a}, \boldsymbol{b}) = (a_0 b_0 \gg 16, \dots)$. Multiply 16-bit unsigned integers and store upper 16 bits of 32-bit product.
`PMADDUBSW`	SSE2	`PMADDUBSW`$(\boldsymbol{a}, \boldsymbol{b}) = ((a_0 b_0 + a_1 b_1), \dots)$. Inner product of two 8-bit signed-unsigned pairs, accumulated into 16-bit results.
`PMADDWD`	SSE2	`PMADDWD`$(\boldsymbol{a}, \boldsymbol{b}) = ((a_0 b_0 + a_1 b_1), \dots)$. Inner product of two signed 16-bit pairs, accumulated into 32-bit results.
`VPDPBUSD`	VNNI	`VPDPBUSD`$(\boldsymbol{c}, \boldsymbol{a}, \boldsymbol{b}) = ((c_0 + a_0 b_0 + a_1 b_1 + a_2 b_2 + a_3 b_3), \dots)$. Inner product of four 8-bit signed-unsigned pairs, accumulated into 32-bit results. Emulated by SSE2: `PMADDUBSW`, `PMADDWD`, `PADDD`.

matrix using $\tilde{\boldsymbol{s}}\tilde{\boldsymbol{s}}^\top$, and rearrange the order of the coefficients of the matrix using W. Note here that since P_i and $\tilde{\boldsymbol{s}}\tilde{\boldsymbol{s}}^\top$ are symmetric matrices, we extract only the upper triangular components of the coefficients, and double the coefficient value if $j \neq k$. As a result, the coefficient vector $\boldsymbol{u}$ has the same size as the coefficient vector of $\bar{P}_i$. Finally, we can obtain $\boldsymbol{s}^\top P_i \boldsymbol{s}$ by calculating the inner product.

Since the size of $\boldsymbol{s}$ is not large, the coefficient vector $\boldsymbol{u}$ can be created quickly. Once $\boldsymbol{u}$ is obtained, it can be reused to calculate $\boldsymbol{s}^\top P_i \boldsymbol{s}$ for all i, which reduces the dominant calculation cost by approximately a factor of ℓ compared to simply calculating $\mathsf{Extr}_W(\tilde{\boldsymbol{s}}^\top \bar{P}_i \tilde{\boldsymbol{s}})$. Moreover, the coefficients regenerated by rejection sampling (public-key coefficients) and the coefficient vector $\boldsymbol{u}$ share the same ordering, so we can compute the inner product directly without any memory reordering.

4 Architecture-level acceleration methods

This section presents several architecture-level acceleration methods leveraging x86-specific instructions. The primary idea is to apply SIMD instructions to major operations of QR-UOV. In the QR-UOV implementation, each calculation is basically performed in bytes since q fits into 8 bits. On the other hand, the x86 architecture has several multiplication instructions for integers, as shown in Table 5. Accordingly, in calculations on $\mathbb{F}_q$, the size of the 8-bit value is appropriately expanded to larger bits, such as 16 bits and 32 bits, and then performed using SIMD integer arithmetic instructions. The remainder is finally taken to return it to 8-bit width.

We first introduce a method for reduction modulo q using SIMD vectors, followed by an approach for matrix multiplication over $\mathbb{F}_q$ and $\mathbb{F}_{q^\ell}$ using SIMD instructions. Next, we describe a memory allocation optimization, which plays a critical role in improving performance. In addition, we present SIMD-based methods to accelerate both evaluation and interpolation in the EMI method (Section 3.1) and rejection sampling during key expansion.

4.1 Reduction modulo q

We perform modular reduction for $\mathbb{F}_q$ arithmetic after the necessary computation, and execute it in parallel across SIMD lanes to improve throughput. We use two methods depending on the input width: the 8-bit case and the wider (16/32-bit) case.

Algorithm 4 SIMD Barrett-style reduction for $q \in \{7, 31, 127\}$ with 15-bit inputs

Input: Packed $x \in [0, 2^{15})$ (16-bit lanes)
Input: Parameters $s \in \mathbb{N}$, $M = \left\lceil \frac{2^{16+s}}{q} \right\rceil$ (both broadcast to lanes)
Output: $r \in [0, q-1]$ with $r \equiv x \pmod{q}$
1: $q' \leftarrow \texttt{PSRLW}(\texttt{PMULHUW}(x, M), s)$ ▷ $q' = \lfloor (x \cdot M)/2^{16+s} \rfloor = \lfloor x/q \rfloor$
2: $u \leftarrow \texttt{PADDW}(x, q')$ ▷ $u = x + q'$
3: $r \leftarrow \texttt{PAND}(u, q)$ ▷ $r = u \bmod (q+1)$ (mask $q = 2^k - 1$)
4: **return** r

For 8-bit inputs, we first apply a partial reduction that exploits the Mersenne form of $q = 2^n - 1$. Let the operator $\gg$ be a right shift and the operator & be a bitwise AND. The upper bits can be folded into the lower n bits as follows:

$$x \equiv (x \,\&\, q) + (x \gg n) \pmod{q}.$$

We then apply one or two conditional subtractions to obtain the result in $[0, q)$.

For wider inputs, we first apply a partial reduction using x86 bytewise inner-product instructions (`PMADDUBSW`, `VPDPBUSD`), and then complete the reduction using a Barrett-style algorithm described in Algorithm 4. For a 16-bit lane written as $a = a_0 + 2^8 a_1$ with $a_0, a_1 \in \{0, \ldots, 255\}$, choose byte weights $b_0 = 1$ and $b_1 \equiv 2^8 \bmod q$ (broadcast to all lanes). A single instruction $\texttt{PMADDUBSW}(a, b) = (a_0 b_0 + a_1 b_1, \ldots)$ performs a per-lane partial reduction modulo q. Similarly, a 32-bit lane is folded with `VPDPBUSD` using byte weights $(1, 2^8, 2^{16}, 2^{24}) \bmod q$.

Algorithm 4 is the Barrett-style reduction method for 15-bit inputs. The idea is to obtain the quotient q' by multiplying a precomputed constant M, and then obtain the remainder using $r = x - q'q$. Related techniques (and compiler implementations) support a wider input domain and may yield a signed residue [GM94, WJ13, CCC+09]. In our implementation, we restrict the input domain to the unsigned 15-bit range. Under this assumption, Algorithm 4 directly returns the canonical remainder $r \equiv x \pmod{q}$ with $r \in [0, q-1]$ using only four x86 SIMD instructions. Both the input and output are processed in 16-bit SIMD lanes. The parameters (q, s, M) are as follows:

$$(q, s, M) = (7, 1, 18725), (31, 3, 16913), (127, 6, 33027).$$

Algorithm 4 is justified as follows. Let $M = \lceil 2^{16+s}/q \rceil = 2^{16+s}/q + \varepsilon$ with $0 \le \varepsilon < 1$, and write $x = aq + b$ with $0 \le b < q$. Then

$$\frac{Mx}{2^{16+s}} = a + \frac{b}{q} + \varepsilon \cdot \frac{x}{2^{16+s}}.$$

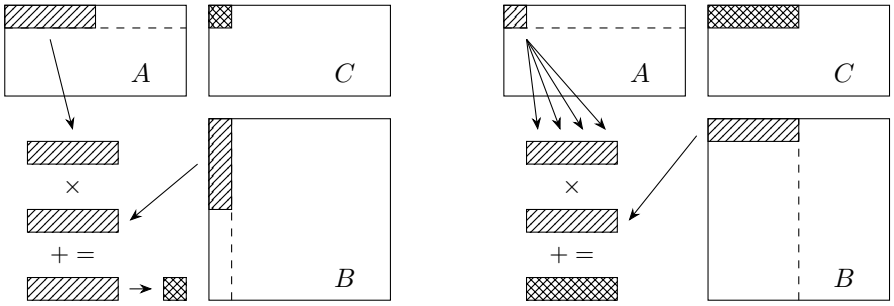
With the parameters above and $x < 2^{15}$, we have

$$\frac{b}{q} + \varepsilon \cdot \frac{x}{2^{16+s}} < \frac{q-1}{q} + \varepsilon \cdot \frac{1}{2^{1+s}} < 1,$$

hence $q' = \lfloor (Mx)/2^{16+s} \rfloor = a$. Finally, $u = x + q' = a(q+1) + b$, so taking $u \bmod (q+1)$ (via masking with $q = 2^k - 1$) returns $b = x \bmod q$.

4.2 Matrix multiplication over $\mathbb{F}_q$ and $\mathbb{F}_{q^\ell}$

In the KeyGen and Sign procedures of QR-UOV, matrix–matrix and matrix–vector multiplications over $\mathbb{F}_{q^\ell}$ constitute the primary computational bottlenecks. In addition, the



(a) Inner product (b) Element-vector

Figure 1: Matrix multiplication $C = AB$ strategies.

EMI method (Section 3.1) requires matrix multiplications over $\mathbb{F}_q$. In the following, we propose efficient multiplication methods for both $\mathbb{F}_{q^\ell}$ and $\mathbb{F}_q$.

The QR-UOV reference implementations [AFI+24] include two distinct strategies for polynomial multiplication over $\mathbb{F}_{q^\ell}$. The Round 1 implementation employed (1) 64-bit multiplication-based polynomial multiplication: For $\ell = 3$, 24-bit units, and for $\ell = 10$, 16-bit units, are packed into 64-bit registers. A single 64-bit multiplication produces a 128-bit result, enabling the multiplication of quadratic polynomials for $\ell = 3$ or cubic polynomials for $\ell = 10$. However, this approach suffers from the limited parallelism of the 64-bit multiplication instruction and does not fully leverage hardware-level SIMD parallelism, resulting in suboptimal performance. To address this, the Round 2 implementation introduced (2) SIMD-based polynomial multiplication, as illustrated in Figure 1(a), where SIMD multiplication instructions are employed to parallelize the inner-product computations within the matrix multiplication. While this approach achieves significant acceleration compared to (1), the performance difference between AVX2 and AVX-512 remains negligible, indicating that the SIMD instructions are not being utilized to their full potential.

To overcome this limitation, we propose a method that parallelizes the computation of multiple elements of the product matrix. The proposed parallelization scheme is depicted in Figure 1(b). In computing the matrix product AB, both A and B are processed in a row-major memory layout. A small set of elements from A is first loaded and broadcast, after which multiple elements from B are loaded. Parallel multiplications are then performed to accumulate multiple partial sums, followed by modular reduction by q to obtain the result. This element-wise vector multiplication (b) is more efficient than the inner-product-based method (a) for two main reasons: First, in (a), each SIMD multiplication requires two SIMD load instructions, whereas (b) requires only one SIMD load and one scalar load per SIMD multiplication, thereby improving the compute intensity (operations per memory access). Second, (a) necessitates an additional horizontal summation within the SIMD register for each matrix element computation, which is eliminated in (b).

As with other SIMD-based approaches, the proposed method requires rearranging the memory layout of the polynomial matrices to ensure that coefficients of the same degree are stored contiguously. Furthermore, our method employs the `VPDPBUSD` instruction, an 8-bit multiplication instruction available in the VNNI extension of x86 CPUs. Although this instruction is specific to VNNI, it can be efficiently emulated with three AVX2 instructions, delivering competitive performance. Because `VPDPBUSD` performs a four-element inner product, the matrix data must be arranged accordingly for matrix multiplication. Specifically, for the matrix product AB (both in row-major layout), four consecutive elements in the column direction of matrix B must be contiguous in memory. We refer to this memory arrangement as the VNNI layout. An example of the VNNI memory layout for a 4×4

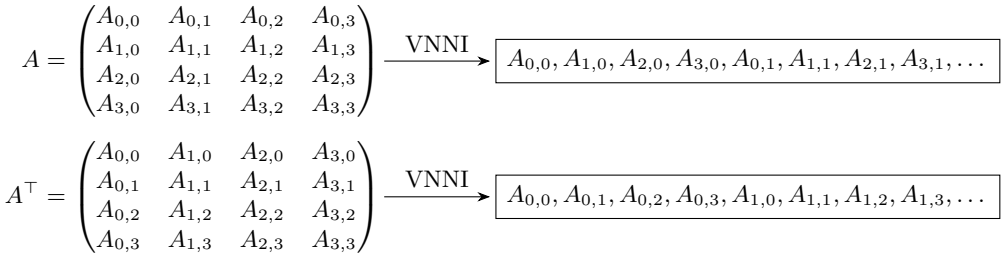


Figure 2: VNNI memory layout of 4×4 block matrix A and A^T.

block matrix A is shown in the upper part of Figure 2. Details of this memory reordering are given in Section 4.3.

Furthermore, in the multiplication over $\mathbb{F}_{q^\ell}$, for each position of multiplication, all the coefficients of both operands A and B (a total of 2ℓ coefficients) are first loaded. The multiplication is then performed degree by degree, and the intermediate results are accumulated into $2\ell - 1$ accumulators. This approach further increases the computational intensity of the operation. Considering the number of SIMD registers available on the x86 architecture (AVX2: 16, AVX-512: 32), when $\ell = 10$, the number of registers becomes insufficient. However, the compiler can automatically spill registers to memory, enabling the computation to proceed with reasonably high performance. Similarly, in multiplication over $\mathbb{F}_q$, the computational intensity is further enhanced by increasing the size of the block matrices processed at once, thereby increasing register utilization. Such techniques are also employed in high-performance computing (HPC) matrix multiplication [GvdG08].

Since the acceleration of the innermost loop in the algorithm is crucial for performance, the proposed implementation employs microkernel functions with varying block matrix sizes, enabling efficient computation of matrix operations of arbitrary sizes by combining these microkernels. In our experiments, `clang` produces faster microkernel code than `gcc`, largely due to fewer register moves. We therefore use `clang` to compile the microkernels by default, noting that different versions or flags may affect this comparison.

4.3 Memory reordering for matrix multiplication

In QR-UOV, matrices are stored in row-major order, and the coefficients of polynomials are provided in a contiguous layout. The symmetric matrix $\bar{P}_{i,1}$ is represented by its upper triangular elements only. High-speed matrix multiplication AB can be achieved by converting both operands to a Structure of Arrays (SoA) layout, i.e., degree-wise coefficient planes, so that each SIMD load fetches only one coefficient degree. In addition, for B we apply VNNI packing within each degree plane, which matches the access pattern of the bytewise inner-product instructions. Since $\bar{P}_{i,1}$ contains a large number of elements in QR-UOV, an efficient expansion of the symmetric matrix is essential.

For this purpose, we leverage the property that the VNNI layout stores columns contiguously for every four rows, and that a 4×4 block matrix A and its transpose $A^\top$ are contiguous and arranged as illustrated in Figure 2. When expanding $\bar{P}_{i,1}$ from its upper-triangular representation, this property enables us to simultaneously unpack and format to VNNI layout, resulting in a fast transformation of the memory layout.

As an example, Figure 3 illustrates the reordering process for $\ell = 3$. By extracting and deinterleaving 4ℓ elements from appropriate positions of the $\bar{P}_{i,1}$, we obtain degree-wise 4×4 block matrices, with each block aligned to a register for that degree. Viewing the symmetric matrix as a collection of 4×4 blocks, each corresponding pair is related by transposition, as shown for A and $A^\top$ in Figure 3. Since the `PSHUFB` instruction performs 16-byte shuffles efficiently, this operation can be implemented with low overhead.

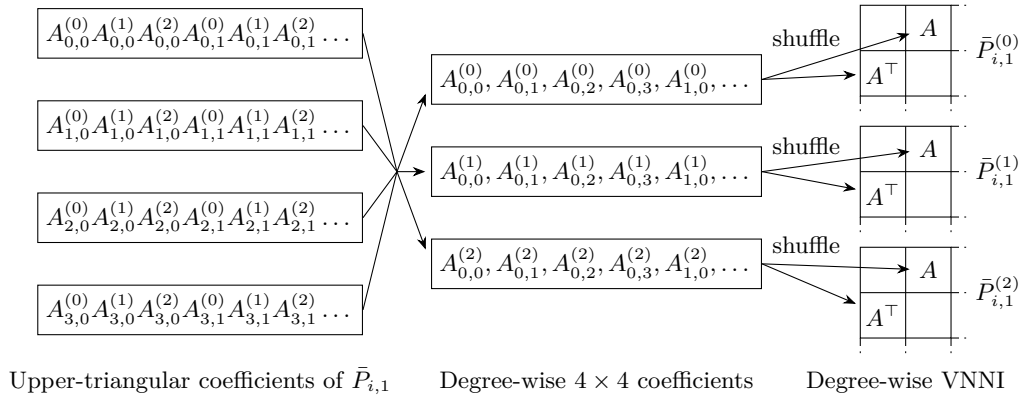


Figure 3: $\bar{P}_{i,1}$ memory reordering from upper-triangular format to degree-wise VNNI.

4.4 Evaluation and interpolation for the EMI method

We describe SIMD-oriented implementations of evaluation and interpolation for the EMI method (Section 3.1). After the memory reordering in Section 4.3, coefficients are stored degree-wise (SoA), so that a single SIMD load fetches only one coefficient degree. We compile the evaluation matrix G and the interpolation matrix U into fixed, branch-free SIMD sequences of lane-wise additions, subtractions, and small-constant multiplications. These sequences execute efficiently on SIMD lanes.

Let $a(x) = \sum_{j=0}^{\ell-1} a_j x^j \in \mathbb{F}_q[x]$ with coefficients $a_0, \dots, a_{\ell-1} \in \mathbb{F}_q$, we write $a(t)$ for its evaluation at $t \in \mathbb{F}_q$ and set $a(\infty) \coloneqq a_{\ell-1}$. For $\ell = 3$ and the set of evaluation points $\{0, 1, 2, -1, \infty\}$, we need the five values $a(0) = a_0,\ a(1) = a_0 + a_1 + a_2,\ a(2) = a_0 + 2a_1 + 4a_2,\ a(-1) = a_0 - a_1 + a_2,\ a(\infty) = a_2$. Using the degree-wise SIMD layout, we compute these values with six modular additions and one subtraction on 8-bit lanes. Small constants (such as 2 and 4) are realized by successive modular doublings.

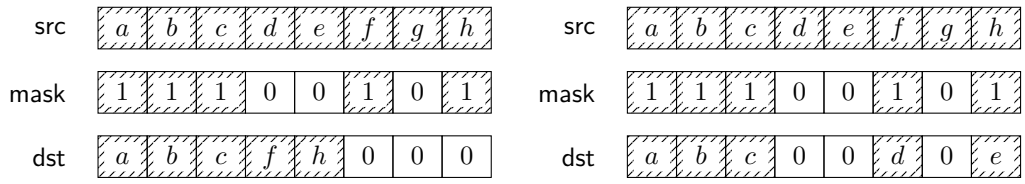
For $(q, \ell) = (31, 10)$ we take the set of evaluation points $\{0, \pm 1, \dots, \pm 8, 9, \infty\}$ ($n_{\text{eval}} = 19$). To exploit the symmetry at $\pm k$, we split $a(x)$ into its even and odd parts. Define $a_{\text{even}}(x) \coloneqq \sum_{j \geq 0} a_{2j} x^{2j},\ a_{\text{odd}}(x) \coloneqq \sum_{j \geq 0} a_{2j+1} x^{2j+1}$. We therefore compute $a_{\text{even}}(k)$ and $a_{\text{odd}}(k)$ once and reuse them to obtain $a(\pm k)$, which reduces the total work for each $\pm k$ pair by roughly a factor of two.

Row scaling can be applied to simplify code generation without changing the algorithm. If the evaluation matrix is pre-scaled as $G' = DG$ with a diagonal $D = \text{diag}(c_1, \dots, c_{n_{\text{eval}}})$, the componentwise product acquires a factor D^2; this is compensated once in interpolation by column scaling $U' = UD^{-2}$. This transformation is independent of how (G, U) was constructed. We did not enable it in the reported benchmarks; we included it as an implementation note because it can simplify code generation and, depending on q and coefficient distributions, may reduce the number of additions by redistributing coefficients across rows.

4.5 Rejection sampling in key expansion

In QR-UOV, rejection sampling is required to obtain uniform elements in $\mathbb{F}_q$ with an odd modulus. The RejSamp procedure splits the PRG output into a head and a tail. In the head, positions equal to q must be replaced by non-q values taken from the tail.

To implement this efficiently with SIMD, we use two data movement operations: Compress and Expand (Figure 4). Compress compacts the selected entries (where mask $= 1$) of a source vector into a dense vector. Expand writes a dense source vector back to the



(a) Compress operation (b) Expand operation

Figure 4: Two suboperations for rejection sampling.

positions specified by a mask. In our implementation, we first apply Compress to the tail to collect its non-q values into a dense buffer. We then mark the positions equal to q in the head and use Expand to fill those positions from the buffer.

With AVX2 instructions, we implement both Compress and Expand as follows. We build a per-byte mask by testing each byte for equality to q and pack it into a bit mask with `PMOVMSKB`. We then apply `PSHUFB` (a byte shuffle) using shuffle-control bytes from a lookup table to perform the required 16-byte permutation. With AVX-512 VBMI2 instructions, `VPEXPANDB` is available for Expand; it operates on up to 64-byte vectors and accelerates the main bottleneck of RejSamp.

In a microbenchmark of RejSamp, our AVX2 and VBMI2 implementations achieve about 8× and 26× speedups, respectively, compared to a scalar C baseline.

5 Performance evaluations

5.1 Experimental conditions

We implemented QR-UOV based on the proposed techniques. This work focuses on performance optimization and does not aim to provide constant-time guarantees. Ensuring constant-time execution requires dedicated side-channel analysis and careful implementation, both of which are beyond the scope of this paper.

We ran benchmarks on Intel Skylake (AVX2) and Intel Sapphire Rapids (AVX-512 VNNI, GFNI, VBMI2). We compiled the implementations primarily with `gcc`. The only exception is the matrix-multiplication microkernels in the proposed QR-UOV implementation, which we compiled with `clang` to improve performance. All experiments were conducted on Amazon EC2 bare-metal instances. The specifications of the machines are summarized in Table 6.

We measured cycle counts using the `libcpucycles` library [Ber24], which utilizes its `amd64-pmc` backend to access hardware performance counters. We disabled Turbo Boost and fixed the CPU to its maximum non-turbo frequency. We pinned the benchmark thread to a fixed core with `taskset` and offlined the Hyper-Threading sibling logical CPU in

Table 6: Benchmark platforms

Property	Skylake	Sapphire Rapids
EC2 instance type	c5n.metal	c7i.metal-24xl
CPU model name	Xeon Platinum 8124M	Xeon Platinum 8488C
Microarchitecture	Skylake-SP	Sapphire Rapids
CPU non-turbo frequency	3.0 GHz	2.4 GHz
Instruction extensions used	AVX2	VNNI, GFNI, VBMI2

Table 7: Benchmarking results of AVX2 implementations on Skylake (Mcycles)

SL	Instance	KeyGen	Sign	Verify
I	QR-UOV $(127, 3)$	3.27	0.87	0.46
	QR-UOV $(31, 3)$	3.87	1.15	0.60
	QR-UOV $(31, 10)$	10.34	5.00	1.76
	QR-UOV $(7, 10)$	28.34	12.28	4.51
	QR-UOV Ref $(127, 3)$	16.74	3.16	2.61
	UOV Ip-pkc+skc	2.96	1.97	0.24
	MAYO1†	0.20	0.57	0.25
III	QR-UOV $(127, 3)$	11.59	2.48	1.38
	QR-UOV $(31, 3)$	14.84	3.47	1.91
	QR-UOV $(31, 10)$	35.79	16.20	5.97
	QR-UOV $(7, 10)$	92.06	36.99	15.04
	QR-UOV Ref $(127, 3)$	62.45	9.59	7.77
	UOV III-pkc+skc	16.45	9.98	1.08
	MAYO3†	0.47	1.26	0.59
V	QR-UOV $(127, 3)$	33.34	6.24	3.64
	QR-UOV $(31, 3)$	36.16	7.52	4.56
	QR-UOV $(31, 10)$	61.73	29.27	12.25
	QR-UOV $(7, 10)$	266.19	91.99	40.53
	QR-UOV Ref $(127, 3)$	211.02	23.86	18.42
	UOV V-pkc+skc	55.69	22.60	2.24
	MAYO5†	1.20	3.04	2.11

† Values from the MAYO specification [BCC+24].

Linux (one hardware thread per core). Each algorithm was executed 1,000 times, and we report median cycle counts. All experiments ran on Ubuntu Linux 24.04 LTS.

As a baseline for comparison, we employed the official Round 2 implementations of QR-UOV[3] [AFI+24] and UOV pkc+skc[4] [BCD+24]. We compiled all SIMD-optimized implementations using the same toolchains and executed them under identical conditions. In addition, Table 7 includes the performance of MAYO as reported in its specification [BCC+24]. These values are provided for reference only and are not directly comparable to our measured cycle counts due to potential differences in the evaluation environment.

5.2 Results

Table 7 shows the results of our AVX2 implementation. Among the four (q, ℓ) parameter sets of QR-UOV, the configuration with $(q, \ell) = (127, 3)$ exhibits the highest performance, whereas the configuration with $\ell = 10$ shows relatively lower performance. Compared to the QR-UOV Round 2 reference implementation, the $(127, 3)$ parameter set at security level (SL) I achieves a speedup of 5.1× in key generation, 3.6× in signature generation, and 5.7× in signature verification. Compared to UOV, the key generation speed is comparable at security level I, while QR-UOV demonstrates higher performance at levels III and V. For signature generation, QR-UOV also outperforms UOV because of the matrix chain optimization described in Section 3.2. On the other hand, signature verification is slower for QR-UOV. The key expansion requires pseudorandom number generation with AES,

[3] https://github.com/qruov/round2 commit `787f8dba` (commit date: 2025-01-10), `avx2` variant
[4] https://github.com/pqov/pqov commit `0ec8862a` (commit date: 2025-08-07), `avx2` and `gfni` variants

Table 8: Benchmarking results of AVX2 and VNNI/GFNI/VBMI2 implementations on Sapphire Rapids (Mcycles)

		AVX2			VNNI/GFNI/VBMI2		
SL	Instance	KeyGen	Sign	Verify	KeyGen	Sign	Verify
I	QR-UOV $(127, 3)$	2.24	0.56	0.29	1.60	0.50	0.27
	QR-UOV $(31, 3)$	2.72	0.75	0.37	2.14	0.68	0.33
	QR-UOV $(31, 10)$	7.33	3.91	1.08	6.41	2.99	0.96
	QR-UOV $(7, 10)$	20.65	9.51	2.69	17.46	7.31	2.40
	UOV Ip-pkc+skc	2.22	1.46	0.15	1.35	1.01	0.15
III	QR-UOV $(127, 3)$	8.26	1.55	0.83	5.70	1.41	0.74
	QR-UOV $(31, 3)$	10.63	2.26	1.15	8.20	2.10	1.02
	QR-UOV $(31, 10)$	26.37	12.53	3.51	22.83	9.67	3.20
	QR-UOV $(7, 10)$	67.12	26.24	9.14	56.98	20.83	8.09
	UOV III-pkc+skc	13.69	8.05	0.63	7.73	5.05	0.63
V	QR-UOV $(127, 3)$	23.93	4.15	2.15	15.75	3.76	1.92
	QR-UOV $(31, 3)$	26.05	4.92	2.65	20.20	4.55	2.38
	QR-UOV $(31, 10)$	45.55	21.64	7.41	39.29	17.23	6.76
	QR-UOV $(7, 10)$	188.15	65.91	23.78	145.37	51.87	21.12
	UOV V-pkc+skc	35.85	19.09	1.37	18.60	8.32	1.31

Table 9: Profiling results of AVX2 implementation of QR-UOV SL-I $(127, 3)$ on Sapphire Rapids (Mcycles)

Component	KeyGen	Sign	Verify
AES-CTR	0.20 (9%)	0.17 (30%)	0.18 (61%)
Rejection sampling	0.07 (3%)	0.07 (12%)	0.08 (27%)†
Matrix multiplication	1.43 (64%)	0.12 (22%)	0.01 (4%)
Evaluation and interpolation	0.25 (11%)‡	—	—
Linear algebra (PLU)	—	0.09 (16%)	—
Others	0.29 (13%)	0.11 (20%)‡	0.02 (8%)
Total	2.24 (100%)	0.56 (100%)	0.29 (100%)

† Including evaluation of the public map.
‡ Including memory ordering.

and the odd modulus introduces additional rejection sampling. Currently, approximately 90% of the total execution time for signature verification comes from the key expansion. For reference, Table 7 also includes the performance of MAYO reported in its specification.

Table 8 shows the performance results on the Sapphire Rapids architecture. The left column shows the results constrained to AVX2 instructions, while the right column shows the results using the available extension instructions. For QR-UOV, performance can be enhanced by employing the VBMI2 instructions (bitwise operations) for key expansion and the VNNI instructions (inner products) for matrix multiplication. In contrast, UOV can leverage the GFNI instructions (multiplication over $\mathbb{F}_{256}$) to accelerate matrix multiplication. When the extension instructions are used, both QR-UOV and UOV exhibit significant performance improvements, achieving comparable acceleration.

Table 9 reports profiling results of AVX2 implementation of QR-UOV $(127, 3)$. It indicates the key generation is dominated by matrix multiplication (64%), whereas signing and verification are primarily limited by the AES component (30% and 61 %, respectively).

6 Conclusion

In this paper, we presented a set of algorithm-level and architecture-level acceleration methods that substantially reduce the computational complexity of core arithmetic operations for QR-UOV software implementation. By exploiting advanced instruction set extensions available in modern x86 processors, such as AVX2 and AVX-512, we demonstrated that the proposed methods enable efficient execution across a range of parameter sets and security levels. In particular, our results showed that for the $(q, \ell) = (127, 3)$ configuration, the proposed implementation achieves a speedup of 5.1× in key generation, 3.6× in signature generation, and 5.7× in signature verification compared to the QR-UOV Round 2 reference implementation. These improvements highlight the practical feasibility of QR-UOV and its competitiveness with UOV, especially at higher security levels.

Our exploration of implementation strategies over small odd prime fields, such as GF(31) and GF(127), further extends the design space for multivariate polynomial signature schemes. By leveraging integer multipliers and recent instruction set extensions, we showed that efficient arithmetic can be achieved even in such field settings.

Future work will investigate the applicability of additional instruction set extensions to further accelerate QR-UOV, and explore implementations on CPUs with different architectures and instruction sets. These directions may lead to broader portability and further performance improvements of multivariate polynomial signature schemes across heterogeneous computing platforms.

A Matrices G and U for $(q, \ell) = (7, 10)$

The evaluation matrix $G \in \mathbb{F}_7^{23\times 10}$ and the interpolation matrix $U \in \mathbb{F}_7^{10\times 23}$ used in the EMI method (see Section 3.1) are listed below. These values are for multiplications in $\mathbb{F}_7[x]/(x^{10} - 2x - 1)$.

$$G = \begin{pmatrix}
1 & 2 & 6 & 0 & 6 & 5 & 6 & 3 & 1 & 5 \\
1 & 2 & 3 & 6 & 2 & 6 & 4 & 2 & 3 & 6 \\
1 & 4 & 4 & 0 & 0 & 4 & 1 & 2 & 2 & 5 \\
1 & 5 & 4 & 2 & 2 & 3 & 5 & 5 & 1 & 0 \\
1 & 0 & 6 & 1 & 4 & 3 & 5 & 2 & 5 & 3 \\
1 & 2 & 1 & 2 & 1 & 1 & 2 & 0 & 4 & 2 \\
1 & 4 & 4 & 6 & 6 & 4 & 1 & 6 & 0 & 5 \\
1 & 6 & 2 & 0 & 3 & 3 & 2 & 1 & 3 & 2 \\
1 & 1 & 4 & 1 & 2 & 3 & 5 & 2 & 1 & 5 \\
1 & 2 & 6 & 3 & 5 & 0 & 6 & 1 & 6 & 6 \\
1 & 6 & 6 & 1 & 2 & 4 & 1 & 4 & 1 & 3 \\
1 & 5 & 2 & 6 & 2 & 0 & 1 & 3 & 0 & 3 \\
1 & 6 & 4 & 3 & 6 & 1 & 0 & 4 & 4 & 6 \\
1 & 4 & 2 & 4 & 6 & 3 & 2 & 6 & 0 & 3 \\
1 & 6 & 2 & 4 & 2 & 6 & 3 & 5 & 0 & 1 \\
1 & 4 & 2 & 1 & 0 & 5 & 4 & 1 & 5 & 5 \\
1 & 4 & 5 & 0 & 2 & 3 & 5 & 5 & 5 & 5 \\
1 & 1 & 3 & 0 & 6 & 3 & 0 & 6 & 1 & 5 \\
1 & 5 & 5 & 6 & 2 & 6 & 6 & 1 & 6 & 3 \\
1 & 6 & 6 & 2 & 0 & 6 & 2 & 5 & 1 & 0 \\
1 & 6 & 1 & 2 & 0 & 3 & 6 & 1 & 1 & 6 \\
1 & 3 & 6 & 6 & 0 & 5 & 5 & 2 & 5 & 4 \\
1 & 0 & 4 & 4 & 5 & 4 & 0 & 1 & 4 & 0
\end{pmatrix}, U = \begin{pmatrix}
4 & 6 & 6 & 2 & 6 & 3 & 5 & 5 & 3 & 6 & 5 & 1 & 0 & 6 & 6 & 4 & 6 & 2 & 5 & 5 & 0 & 4 & 2 \\
3 & 3 & 5 & 6 & 0 & 3 & 5 & 5 & 4 & 6 & 5 & 3 & 1 & 5 & 3 & 0 & 4 & 4 & 4 & 0 & 3 & 0 & 5 \\
0 & 4 & 6 & 3 & 3 & 6 & 3 & 3 & 1 & 2 & 3 & 0 & 3 & 3 & 5 & 0 & 2 & 2 & 5 & 3 & 4 & 1 & 1 \\
6 & 1 & 5 & 3 & 5 & 2 & 6 & 2 & 4 & 2 & 4 & 4 & 6 & 1 & 6 & 2 & 4 & 6 & 4 & 5 & 5 & 3 & 5 \\
0 & 2 & 3 & 2 & 4 & 5 & 6 & 1 & 3 & 2 & 5 & 6 & 4 & 5 & 4 & 4 & 4 & 1 & 6 & 4 & 5 & 6 & 2 \\
2 & 4 & 4 & 6 & 3 & 1 & 2 & 2 & 4 & 3 & 3 & 0 & 4 & 4 & 0 & 3 & 1 & 4 & 5 & 4 & 0 & 0 & 4 \\
6 & 3 & 1 & 2 & 1 & 5 & 4 & 0 & 3 & 0 & 6 & 2 & 1 & 6 & 6 & 0 & 1 & 4 & 1 & 4 & 4 & 6 & 4 \\
0 & 3 & 0 & 5 & 5 & 3 & 3 & 3 & 1 & 6 & 0 & 0 & 5 & 6 & 4 & 1 & 3 & 1 & 0 & 6 & 2 & 5 & 1 \\
4 & 4 & 4 & 1 & 0 & 5 & 1 & 2 & 5 & 1 & 6 & 6 & 4 & 1 & 1 & 1 & 5 & 3 & 4 & 4 & 0 & 6 & 2 \\
2 & 2 & 0 & 1 & 5 & 3 & 5 & 4 & 6 & 3 & 1 & 0 & 6 & 0 & 0 & 0 & 5 & 1 & 6 & 3 & 2 & 4 & 4
\end{pmatrix}.$$

References

[AFI+24] Rika Akiyama, Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Koha Kinjo, Haruhisa Kosuge, Satoshi Nakamura, Shingo Orihara, Tsuyoshi Takagi,

and Kimihiro Yamakoshi. QR-UOV. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCC+24] Ward Beullens, Fabio Campos, Sofía Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCD+24] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. UOV — Unbalanced Oil and Vinegar. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCH+23] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih, and Bo-Yin Yang. Oil and vinegar: Modern parameters and implementations. *IACR TCHES*, 2023(3):321–365, 2023.

[Ber24] Daniel J. Bernstein. libcpucycles. https://cpucycles.cr.yp.to/, 2024.

[BPR+21] S. Ballet, J. Pieltant, M. Rambaud, H. Randriambololona, R. Rolland, and J. Chaumine. On the tensor rank of multiplication in finite extensions of finite fields and related issues in algebraic geometry. *Uspekhi Mat. Nauk*, 76(1(457)):31–94, 2021.

[CC88] David V. Chudnovsky and Gregory V. Chudnovsky. Algebraic complexities and algebraic curves over finite fields. *J. Complex.*, 4(4):285–316, 1988.

[CCC+09] Anna Inn-Tung Chen, Ming-Shing Chen, Tien-Ren Chen, Chen-Mou Cheng, Jintai Ding, Eric Li-Hsiang Kuo, Frost Yu-Shuang Lee, and Bo-Yin Yang. SSE implementation of multivariate PKCs on modern x86 CPUs. In Christophe Clavier and Kris Gaj, editors, *CHES 2009*, volume 5747 of *LNCS*, pages 33–48. Springer, Berlin, Heidelberg, September 2009.

[CHR+16] Ming-Shing Chen, Andreas Hülsing, Joost Rijneveld, Simona Samardjiska, and Peter Schwabe. From 5-pass MQ-based identification to MQ-based signatures. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 135–165. Springer, Berlin, Heidelberg, December 2016.

[CHT12] Peter Czypek, Stefan Heyse, and Enrico Thomae. Efficient implementations of MQPKS on constrained devices. In Emmanuel Prouff and Patrick Schaumont, editors, *CHES 2012*, volume 7428 of *LNCS*, pages 374–389. Springer, Berlin, Heidelberg, September 2012.

[CKY21] Tung Chou, Matthias J. Kannwischer, and Bo-Yin Yang. Rainbow on Cortex-M4. *IACR TCHES*, 2021(4):650–675, 2021.

[CLP+18] Ming-Shing Chen, Wen-Ding Li, Bo-Yuan Peng, Bo-Yin Yang, and Chen-Mou Cheng. Implementing 128-bit secure MPKC signatures. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 101-A(3):553–569, 2018.

[DCP+20] Jintai Ding, Ming-Shing Chen, Albrecht Petzoldt, Dieter Schmidt, Bo-Yin Yang, Matthias J. Kannwischer, and Jacques Patarin. Rainbow. Technical report, National Institute of Standards and Technology, 2020. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions.

[FIH+23] Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Tsuyoshi Takagi, Kan Yasuda, Toshiyuki Miyazawa, Tsunekazu Saito, and Akira Nagai. QR-UOV. Technical report, National Institute of Standards and Technology, 2023. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[FIKT21] Hiroki Furue, Yasuhiko Ikematsu, Yutaro Kiyomura, and Tsuyoshi Takagi. A new variant of unbalanced Oil and Vinegar using quotient ring: QR-UOV. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 187–217. Springer, Cham, December 2021.

[GM94] Torbjörn Granlund and Peter L. Montgomery. Division by invariant integers using multiplication. In Vivek Sarkar, Barbara G. Ryder, and Mary Lou Soffa, editors, *Proceedings of the ACM SIGPLAN'94 Conference on Programming Language Design and Implementation (PLDI), Orlando, Florida, USA, June 20-24, 1994*, pages 61–72. ACM, 1994.

[GvdG08] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Trans. Math. Softw.*, 34(3):12:1–12:25, 2008.

[Hwa24] Vincent Hwang. A survey of polynomial multiplications for lattice-based cryptosystems. *CiC*, 1(2):1, 2024.

[KPG99] Aviad Kipnis, Jacques Patarin, and Louis Goubin. Unbalanced Oil and Vinegar signature schemes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 206–222. Springer, Berlin, Heidelberg, May 1999.

[Ran12] Hugues Randriambololona. Bilinear complexity of algebras and the chudnovsky-chudnovsky interpolation method. *J. Complex.*, 28(4):489–517, 2012.

[SCH+19] Simona Samardjiska, Ming-Shing Chen, Andreas Hulsing, Joost Rijneveld, and Peter Schwabe. MQDSS. Technical report, National Institute of Standards and Technology, 2019. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions.

[WCD+24] Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Jan Adriaan Leegwater, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. SNOVA. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[WJ13] Henry S. Warren Jr. *Hacker's Delight, Second Edition*. Pearson Education, 2013.